FOORT: Flexible Object-Oriented Ray Tracing

DOCUMENTATION

Daniel R. Mayerson[1,2], Seppe J. Staelens[3,2], Fabio Bacchini[4,5,2]

[1] Institute for Theoretical Physics, KU Leuven, Celestijnenlaan 200D, B-3001 Leuven, Belgium
[2]Leuven Gravity Institute, KU Leuven, Celestijnenlaan 200D box 2415, 3001 Leuven, Belgium
[3] DAMTP, Centre for Mathematical Sciences, University of Cambridge, Wilberforce Road, Cambridge CB3 0WA, United Kingdom
[4] Centre for mathematical Plasma Astrophysics, KU Leuven, Celestijnenlaan 200B, B-3001 Leuven, Belgium
[5] Royal Belgian Institute for Space Aeronomy, Solar-Terrestrial Centre of Excellence, Ringlaan 3, 1180 Uccle, Belgium

daniel.mayerson@kuleuven.be, ss3033@cam.ac.uk, fabio.bacchini@kuleuven.be

Welcome to the documentation for `FOORT`, which will make using `FOORT` but a small efFOORT!

This documentation should be considered complementary to the `FOORT` paper, where the ray-tracer and its underlying physics is presented. This file serves to give an overview of all possible options currently implemented in `FOORT` (and which can be set in a `FOORT` configuration file), as well as an explanation of the necessary procedures to add new objects (`Metric`, `Diagnostic`, and `Termination`) to the `FOORT` code itself.

FOORT version: 1.0.0. Documentation compiled: Tuesday 10th November, 2026

Contents

1 Installation and Compilation **1**

2 Configuration Options **1**
2.1 `Metric` . . . 1
2.2 `Source` . . . 3
2.3 `Diagnostics` . . . 3
2.4 `Terminations` . . . 6
2.5 `ViewScreen` . . . 8
2.6 `Integrator` . . . 9
2.7 `Output` . . . 10

3 Processing Output **11**
3.1 Output files . . . 11
3.2 Setting up the Python modules . . . 13
3.3 Images with `pyFOORT` . . . 13
3.4 Visibility amplitudes with `pyFOORTInterferometry` . . . 15
3.5 Example . . . 16

4 Procedures to Add New Objects **17**
4.1 Adding a new Metric . . . 17
4.2 Adding a new Diagnostic . . . 18
4.3 Adding a new Termination . . . 19

1 Installation and Compilation

Please see the README.md file in the root of the repository for the most up-to-date installation and compilation (with CMake) information. This includes information on fixes necessary for older `gcc` compilers (such as `gcc` 7.5).

2 Configuration Options

`FOORT` is set up that all possible parameters that can be changed, can be changed from the configuration file only — meaning recompiling is unnecessary unless you are adding new objects (e.g. a new `Metric`) to the code.

Always be mindful of options that take numbers: if they take an `int`, then the number passed must be a integer number — e.g. 1000.0 will not be recognized. (However, if a floating-point number (`real`) is expected, `FOORT` will recognize and automatically convert an `int`.) If the incorrect format of number is passed, then this option will not be recognized (and the default value(s) will be used).

All Setting names (including the `Diagnostic` and `Termination` headings) are case-sensitive. The quoted names that select an object (the `Metric` and `Source Name`, `Mesh` and `Integrator Type`, `EmissionModel` and `FluidVelocityModel`) are case-insensitive. File names and paths are used exactly as given (i.e. case-sensitive).

2.1 `Metric`

This heading is where the metric is selected and all its options set. All metrics can carry the option:

- `RLogScale = bool;`
  where `bool` is `true` or `false`. If `true`, then the metric will be evaluated in the logarithmic radial coordinate $u = \log r$; this usually leads to better and faster integration. (Default: `true`.)

The type of metric is set with the following option:

- `Name = "MetricName";`
  where MetricName is one of the following:
  - `Kerr`: Kerr in normal Boyer-Lindquist coordinates.
  - `FlatSpace`: flat space in spherical coordinates.
  - `Rasheed-Larsen`: Rasheed-Larsen black hole with electric and magnetic charges; parameters rescaled such that $M = 1$.
  - `Johannsen`: The Johannsen black hole with $M = 1$ and the four leading order deviation parameters turned on.
  - `Manko-Novikov`: The Manko-Novikov black hole with $M = 1$ and an octopole bump turned on.
  - `Kerr-Schild`: The Kerr metric with $M = 1$ in Kerr-Schild (spherical) coordinates.
  - `ST3Cr`: The horizonless ring fuzzball metric with charge parameters P, q_0, and scaling parameter λ. Note: mass is *not* rescaled for this metric!
  - `BosonStar`: A static, spherically symmetric (ultracompact) solitonic boson star, interpolated from numerical data files; `FOORT` includes one example model (see the `FOORT` paper for details). Note: mass is *not* rescaled for this metric!

  By default (if this setting is not specified in the configuration), the Kerr metric will be selected.

Depending on which metric has been selected, there can be additional options to be specified:

- `Kerr` or `Kerr-Schild`:
  - `a = real;`
    where `real` is the dimensionless spin (a/M) of the black hole, so must be a number between -1 and 1; FOORT will give a warning if this is not satisfied. (Default: 0.5.)
  - `m = real;` (`Kerr` only)
    Mass of the Kerr black hole. (Default: 1.0.)
- `FlatSpace`: No further options to specify.
- `Rasheed-Larsen`:
  - `m = real; a = real; p = real; q = real;`
    These specify the parameters for the Rasheed-Larsen black hole metric. Note that the parameters must satisfy $p, q \geq 2m$, $a^2 < m^2$ and $m > 0$; FOORT will give a warning if this is not satisfied. The parameters (which all have dimensions of length) will always be rescaled so that the mass of the black hole is $M = 1$ (i.e. they are measured in units of the black hole mass); note that $M = (p+q)/4$. The horizon is at $r_+ = m + \sqrt{m^2 - a^2}$. (Defaults: $m = 1, a = 0.5, p = q = 2$.)
- `Johannsen`:
  - `a = real;`
    This sets the rotation parameter; just like for Kerr, $|a| < 1$ since $M = 1$. (Default: 0.7.)
  - `alpha13 = real; alpha22 = real; alpha52 = real; epsilon3 = real;`
    This sets the four leading order parameters that control deviations from Kerr. (When all of these are zero, the metric is exactly Kerr.) The allowed region of these variables is a complicated non-linear relation; for each parameter, FOORT only checks the relation that must hold when the other parameters vanish. (Defaults: all zero except $\alpha_{13} = 2.0$.)
- `Manko-Novikov`:
  - `a = real;`
    This sets the rotation parameter; just like for Kerr, $|a| < 1$ since $M = 1$. (Default: 0.)
  - `alpha3 = real;`
    This sets the octupolar deviation $M_3/M^4 \sim \alpha_3$ from Kerr (so $\alpha_3 = 0$ corresponds to Kerr). (Default: 5.0.)
- `ST3Cr`:
  - `P = real; q0 = real;`
    Charge parameters for the ring fuzzball. Note: the mass of this object (which does not get rescaled) is $M = (3P + q_0 - P^3)/2$. (Defaults: $P = 2.0, q_0 = 50.0$.)
  - `lambda = real;`
    The scaling parameter for the ring fuzzball. (Default: 0.19.)

  `FOORT` will check if the given values of P, q_0, λ are in the allowed range.
- `BosonStar`:

– `PhiFilename = "filename"; MFilename = "filename";`
The two data files containing $\Phi(r) = \log \alpha_{\rm BS}(r)$ and the mass function $m(r)$, respectively, where the metric is $\mathrm{d}s^2 = -e^{2(\Phi(r)-\Phi_\infty)}\mathrm{d}t^2 + (1-2m(r)/r)^{-1}\,\mathrm{d}r^2 + r^2\mathrm{d}\Omega^2$. Each file has two columns (r and the function value), and both files must use the same r grid. The files are always read from the folder `BosonStar/`, relative to the directory from which `FOORT` is run. (Defaults: `"Phi.dat"` and `" m.dat"`, the example model included with `FOORT`.)

– `PhiInfinity = real;`
The asymptotic value Φ_∞ of $\Phi(r)$, subtracted so that the lapse tends to 1 at infinity. (Default: 1.3890394812, the value for the included model.)

– `NumLines = int;`
The number of lines read from the top of each data file; must not exceed the file length. (Default: 11087, all lines of the included files.)

For the included example model, radii are measured in units of $1/\mu_{\rm BS}$, and the boson star has mass $M \approx 10.42$ in these units; choose the camera position, screen size and `SphereRadius` accordingly. The metric has no horizon, so the `Horizon` Termination will be turned off automatically. When using `EquatorialEmission` with the `GeneralCircularRadial` fluid model, the ISCO search returns the outermost transition from stable to unstable circular orbits, i.e. the outer marginally stable orbit ($r \approx 62.5$ for the included model); the inner region of stable orbits ($r \lesssim 28$) is ignored. The default `ISCOLowerBound` and `ISCOUpperBound` suffice.

2.2 `Source`

This heading is where you determine a source for the geodesic equation (i.e. a force on the right-hand side for the geodesic equation, making the geodesic fail to be an actual geodesic). Currently, only `NoSource` is implemented (i.e. no rhs to the geodesic equation):

- `Name = "SourceName";`
where currently SourceName can only be `NoSource` (and there are no further options to specify).

2.3 `Diagnostics`

Inside the heading `Diagnostics`, each `Diagnostic` has its own subheading. Under their own subheading, the most important options to specify is:

- `On = bool;`
where `bool` is `true` is you want the Diagnostic to be turned on and `false` otherwise. Only if the Diagnostic is turned on will the other options in the Diagnostic's subheading be considered.

- `UseForMesh = bool;`
where `bool` is `true` if this is the Diagnostic that should be used in the Mesh to determine "values" to assign to geodesics and "distances" between neigboring geodesics, which the Mesh will use to determine where to refine. Note: only one Diagnostic can have this setting set to `true`!

By default, if no diagnostic sets `UseForMesh` to `true`, `FourColorScreen` will be used if it is turned on. If `FourColorScreen` is not turned on, then all Diagnostic settings revert to the default of only turning on `FourColorScreen`. If more than one Diagnostic has `UseForMesh` set to `true`, the first one to set it will be used (in the order of Diagnostics given below).

The different possible Diagnostic subheadings and further options therein are:

- `FourColorScreen`: This is the four-color screen, assigning an integer 1-4 to the geodesic according to which quadrant of the boundary sphere it has escaped to. Geodesics that do not terminate due to exiting the boundary sphere (due to e.g. timing out or falling into a horizon) will be assigned 0. There are no other settings that `FourColorScreen` takes.

- `GeodesicPosition`: This keeps track of the position of the geodesic along its integration and outputs all those positions; this can be used for e.g. plotting individual geodesic trajectories. The options to specify are:

  - `UpdateFrequency = int;`
    with `int` a number larger or equal to zero. The Diagnostic will only store a position every number of steps as indicated here. If this is 0, the Diagnostic does not update the position during integration, but may update at start and finish according to the following two options. (Default is 1.)
  - `UpdateStart = bool;`
    (Only relevant if `UpdateFrequency = 0`.) Decides whether the Diagnostic stores the position of the geodesic at the very start of integration. (Default: `false`.)
  - `UpdateFinish = bool;`
    (Only relevant if `UpdateFrequency = 0`.) Decides whether the Diagnostic stores the position of the geodesic at the very end of integration. (Default: `true`.)
  - `OutputSteps = int;`
    Here, `int` is the (approximate!) number of positions per geodesic that will be outputted to file. If the Diagnostic has stored more than this many positions during the integration of the geodesic, the Diagnostic will select only (approximately) this many steps to output to file, evenly selected among the stored positions. If this is set to 0, it outputs all steps. (Default is 0.)

- `EquatorialPasses`: This keeps track of the number of passes through the equator $\theta = \pi/2$ that the geodesic makes. It returns a positive number for geodesics that have escapes out the boundary sphere, and a negative number for geodesics that eventually fall in the horizon. This Diagnostic also takes the options `UpdateFrequency`, `UpdateStart`, `UpdateFinish` just as `GeodesicPosition` does above (but with defaults `UpdateStart = true`, `UpdateFinish = true`). In addition, it takes the option:

  - `Threshold = real;`
    Only consider a geodesic "above" or "below" the equatorial plane if its θ coordinate is `Threshold`$\times\pi/2$ above or below $\pi/2$. This threshold is mainly useful for (near-)equatorial camera positions, since in that case without a threshold there is often a discontinuity in equatorial passes between the top and bottom half of the screen. (Default is 0.01.)

- `ClosestRadius`: This keeps track of the smallest radius r that the geodesic reaches. The true r value is always reported, even if a logarithmic r scale is used in integration. Note that if a geodesic enters a horizon, it will reported to reach $r = 0$. This Diagnostic only takes the options `UpdateFrequency` (Default: 1), `UpdateStart`, `UpdateFinish` (see `GeodesicPosition` above, but defaults are `UpdateStart = true`, `UpdateFinish = true`).

- `EquatorialEmission`: This keeps track of the brightness intensity of the geodesic assuming a particular model for emission from an equatorial disc. It will output both the brightness intensity and the number of equatorial passes (just as `EquatorialPasses` does). Note that `EquatorialEmission`, if turned on, automatically turns off `EquatorialPasses` if it is turned on, and additionally assumes the role of value mesh diagnostic if `EquatorialPasses`

had this role. The total brightness intensity $I_{\rm tot}$ for the geodesic is calculated as:

$$I_{\rm tot} = \sum_{n=1}^{N_{max}} \zeta_n I_0(p^{(n)}) g(p^{(n)})^f, \tag{2.1}$$

where the sum is over the n passes through the equatorial plane that the geodesic makes, $\zeta_1 = 1, \zeta_{n>1} = \zeta$ is the "geometric fudge factor" (used to mimic the effects of a geometrically thick disc), N_{max} is either the total number of passes that the geodesic makes or a pre-specified maximum allowed passes (this can be set to a finite number to avoid partially unresolved higher order rings), $I_0(p^{(n)}$ is the local emitted intensity, $g(p^{(n)})$ is the redshift factor (determined by the dot product of the velocity of the emitting fluid and the velocity of the geodesic at that point), and f is the power of the redshift factor (3 or 4). To specify all of these degrees of freedom, `EquatorialEmission` takes the same options as `EquatorialPasses` listed above, and additionally:

- `GeometricFudgeFactor = real;`
  This specifies the geometric fudge factor (ζ). (Default is 1.0.)
- `EquatPassUpperBound = int;`
  The maximum number of equatorial passes allowed to contribute to the intensity (N_{max}). Setting this to 0 means unlimited passes allowed. (Default is 0.)
- `RedshiftPower = int;`
  The power of the redshift used in calculating the observed intensity (f); should be 3 or 4. (Default is 3.)
- `EmissionModel = "EmissionModelName";`
  Model for local emitted intensity $I_0(p)$. Depending on which model used, could have additional options to set as well. Currently, there is only one option for `EmissionModelName`:
  - `GLMJohnsonSU`: Johnson's SU model with three parameters as used by GLM:

$$I_0(t, r, \theta, \phi) = \left((r-\mu)^2 + \sigma^2\right)^{-1/2} \exp\left(-\frac{1}{2}\left[\gamma + \operatorname{arcsinh}\left(\frac{r-\mu}{\sigma}\right)\right]^2\right). \tag{2.2}$$

    This profile only depends on the radius. Additional options to specify are:
    - `mu = real;`
    - `gamma = real;`
    - `sigma = real;`

    These specify the three parameters used in the model. (Default for μ is the horizon radius (if a black hole metric) or 1.0 otherwise; other defaults are $\gamma = 0.0$ and $\sigma = 1.0$.)
- `FluidVelocityModel = "FluidVelocityModelName";`
  Model used to determine the local fluid four-velocity (which is used to determine the redshift factor). Depending on which model used, could have additional options to set as well. Currently, there is only one option for `FluidVelocityModelName`:
  - `GeneralCircularRadial`: A "mix" of (sub)Keplerian circular orbits and infalling (geodetic) orbits. Additional options to specify are:
    - `xi = real;`
    - `betar = real;`
    - `betaphi = real;`
    - `ISCOLowerBound = real;`
    - `ISCOUpperBound = real;`

The three fluid parameters ξ, β_r, β_ϕ must lie between 0.0 and 1.0. (`FOORT` will check and rescale the inputted values if necessary.) ξ is the sub-Keplerian parameter; for $\xi = 1.0$ the pure (geodesic) Keplerian circular orbits are used. β_r and β_ϕ determine the "mix" between the circular orbits and the radial infall orbits; for $\beta_r = \beta_\phi = 1.0$ purely the circular orbits are used, and for $\beta_r = \beta_\phi = 0.0$ the pure radial infall is used. (Defaults are $\xi = \beta_r = \beta_\phi = 1.0$.) The parameters `ISCOLowerBound` and `ISCOUpperBound` set the search range for the ISCO search, in true radii. (Defaults: for a metric with horizon, this is the horizon radius and $10\times$ the horizon radius; otherwise, 0.05 and 1000.) Note: The `GeneralCircularRadial` assumes the metric is stationary and axisymmetric (as it assumes p_t, p_ϕ are conserved); equatorially reflection symmetric (or at least, that timelike geodesics confined to the equatorial plane exist); and that the metric is diagonal in r and θ and block-diagonal in (t, ϕ) (so-called circular metrics can always be written in such a form). Use caution when using this fluid model on metrics that violate these assumptions, as unphysical behaviour can occur; for example, using this fluid model with `Kerr-Schild` (with e.g. $g_{rt} \neq 0$) will display (incorrectly) different emission than with the `Kerr` metric.

2.4 `Terminations`

Terminations are set very similar to Diagnostics as discussed above. Each Termination has its own subheading, with most important option to specify:

- `On = bool;`
  which determines if the Termination is used or not.

By default (e.g. if no Terminations are turned on, or there is no `Terminations` subheading), `TimeOut` and `BoundarySphere` will be turned on with the default options set as indicated below.

The different possible Termination subheadings and further options therein are:

- `Horizon`: If the metric has a horizon at a constant r radius, then we want to stop integration before we get to the horizon, since usually the horizon is a coordinate singularity where geodesics spend an infinite amount of coordinate time approaching. (FOORT will check whether the `Metric` is a descendant class of `SphericalHorizonMetric`; if not, this Termination will be turned off.) Options to set are:
  - `Epsilon_Horizon = real;`
    Here, `real` is the fraction of the horizon radius at which to terminate the geodesic above the horizon radius. For example, if $r = r_h$ is the horizon and `real` $=\epsilon$, then integration will be terminated at $r = r_h(1 + \epsilon)$. By default, this is set to 0.01.
  - `UpdateFrequency = int;`
    Check this condition every `int` steps of the geodesic integration. By default, this is set to 1.

- `GeneralSingularity`: If the metric has an arbitrary number of arbitrary codimension singularities, then this termination can be used to terminate integration at a distance from the singularity or singularities. (FOORT will check whether the `Metric` is a descendant class of `SingularityMetric`; if not, this Termination will be turned off.) Options to set are:
  - `Epsilon = real;`
    This is the minimum (flat, coordinate) distance from the singularity that the geodesic is allowed to reach before terminating. Note: this is calculated as a completely flat distance, not taking into account spherical coordinates; e.g. the

distance d is calculated as $d^2 = (r-r_0)^2+(\theta-\theta_0)^2$ for a singularity at $r = r_0, \theta = \theta_0$. (Default: 0.001.)

- `ConsoleOutput = bool;`
  If `true`, then a console message is outputted every time a geodesic hits a singularity. (Default: `false`.)
- `UpdateFrequency = int;`
  Check this condition every `int` steps of the geodesic integration. By default, this is set to 1.

- `BoundarySphere`: A (fictitious) boundary sphere at a constant radius far away from the object; if the geodesic escapes *outside* this sphere, we stop integration. Options to set:
  - `SphereRadius = real;`
    This radius `real` is the large radius at which to terminate integration. By default, set to 1000.
  - `UpdateFrequency = int;`
    This is analogous to the same setting under `Horizon` above. (Default is 1.)

- `TimeOut`: Terminate the geodesic integration if it has taken too many integration steps. This sometimes happens if a geodesic travels too close to a coordinate singularity. Options to set are:
  - `MaxSteps = int;`
    The maximum number of integration steps allowed before the geodesic terminates. Default is 10000. Note: make sure to set this higher (e.g. 100000 or more) when not using `RLogScale = false` in the metric, as otherwise most geodesics will time-out prematurely!
  - `UpdateFrequency = int;`
    Analogous to the same setting under `Horizon` above. Default is 1.
    Warning: if this is not set to 1, then the effective maximum number of integration steps that the geodesic is allowed before being terminated is `MaxSteps`$*$`UpdateFrequency`!

- `ThetaSingularity`: Terminate the geodesic integration if it comes too close to a polar coordinate singularity ($\theta = 0, \pi$). This can sometimes be useful to check in cases where geodesics are pushed towards the poles in extreme metrics (e.g. Manko-Novikov). Options to set:
  - `Epsilon = real;`
    How close θ is allowed near the poles. (Default is 10^{-5}.)
  - `UpdateFrequency = int;`
    This is analogous to the same setting under `Horizon` above. (Default is 1.)

- `NaN`: Terminate the geodesic integration if there is a `NaN` ("not a number") encountered in at least one of the components of the current geodesic position or velocity. This is useful to turn on if `NaN`'s are suspected to turn up in the metric evaluation (for example, when debugging a new `Metric`): without this `Termination` turned on, `NaN`'s in the geodesic position or velocity will typically ensure the geodesic keeps integrating pointlessly until it times out with the `TimeOut Termination`. Options to set are:
  - `ConsoleOutput = bool;`
    If `true`, then there will also be a console message whenever a `NaN` is encountered; this can be useful for debug purposes. (Default is `true`.)
  - `UpdateFrequency = int;`
    Analogous to the same setting under `Horizon` above. (Default is 1.)

2.5 `ViewScreen`

Under this subheading, all options are set that determine the location, size, orientation, etc. of the viewing screen. The camera is assumed to be a spherical point-sized camera with a certain aperture; the "screen size" is then determined by this aperture — the "screen" is taken to be an square in the plane going through $r = 0$ and perpendicular to the viewing direction.

The options that can be specified are:

- `Position = { t = real; r = real; theta = real; phi = real; }`
  This specifies the position of the camera. (Default is $(0, 1000, \pi/2, 0)$.)
  Warning: If the initial radius of the camera is *outside* of the boundary sphere radius set in the `BoundarySphere` Termination, then all geodesics will terminate immediately after zero steps!

- `Direction = { t = real; r = real; theta = real; phi = real; }`
  This specifies the direction that the camera is pointing in.
  Warning: Currently, this option is **ignored**; the camera is *always* pointing straight towards $r = 0$ (i.e. the direction vector is $(0, -1, 0, 0)$).

- `ScreenSize = { x = real; y = real; }`
  This sets the screen size. The screen is centered around $r = 0$. For example, when the Kerr metric is selected, units are such that $M = 1$; when $a = 0$, the shadow of the Schwarzschild black hole is a circle with radius $\sqrt{27}$[1] on the viewing screen. (Default: $(10, 10)$.)

- `ScreenCenter = { x = real; y = real; }`
  This sets the center of the screen. If this is the origin $(0, 0)$, then the center of the screen is determined by following the camera direction given in `Direction` as taken from the camera position given in `Position`; any non-zero `ScreenCenter` simply offsets the center of the screen from this point. (Default: $(0, 0)$.)

Finally, there is a subheading `Mesh`, under which the Mesh settings should be set. The Mesh is what determines which pixels on the viewing screen are integrated. The options under the `Mesh` subheading are:

- `Type = "MeshName";`
  This indicates the type of Mesh to be used. The possibilities for MeshName are:
  - `SimpleSquareMesh`: This is a square mesh with equal number of pixels in length and width, and pixels evenly distributed over the viewing screen.
  - `InputCertainPixelsMesh`: On a square mesh (such as defined in `SimpleSquareMesh`), have the user input some pixels to be integrated. This can be useful for e.g. integrating only a few geodesics with the Diagnostic `GeodesicPosition` turned on, to be able to follow and plot these few geodesics along their trajectories.
  - `SquareSubdivisionMesh`: This Mesh starts in a first integration iteration with a simple square mesh (such as in `SimpleSquareMesh`), and then further subdivides certain squares into smaller squares in following iterations. It uses the Diagnostic that has `UseForMesh` turned on (see above) to determine "values" and "distances" between geodesics/pixels, which is used to determine which pixel squares to subdivide.

When no Mesh is set with the `Type` option, the default is a `SimpleSquareMesh` (with additional default options as listed below).

Depending on which Mesh is selected with the `Type` option, there are different additional options to set:

[1] Up to rescalings of $\mathcal{O}(1/r_{\text{cam}})$, see the `FOORT` paper for further details.

- `SimpleSquareMesh`:
  - `TotalPixels = int;`
    This specifies the total number of pixels in the entire grid. If this is not a perfect square, it will be rounded downwards to the nearest perfect square (because each row and column must have an integer number of pixels). (Default: 10000.)
- `InputCertainPixelsMesh`:
  - `TotalPixels = int;`
    This is the same option as for `SimpleSquareMesh`, since the user will be inputting pixels to integrate which live on such a square grid. (Default: 10000.)
- `SquareSubdivisionMesh`:[2]
  - `InitialPixels = int;`
    This is similar to the option `TotalPixels` for `SimpleSquareMesh`: the option `InitialPixels` determines the initial square (equally spaced) pixels to be integrated in the first integration loop. (Again, if it is not a perfect square, it will be rounded down to the nearest perfect square.) (Default: 100.)
  - `MaxPixels = int;`
    This is the maximum number of pixels that can be integrated *in total*, i.e. over *all* integration loops combined. If this is set to 0, then there is no maximum number (integration will continue until the maximum subdivision has been reached or every pixel's weight is zero). (Default: 100.)
  - `MaxSubdivide = int;`
    This sets the maximum number of times a square of pixels can be subdivided. Note that the initial grid has subdivision level 1. (Default: 1, which is also what will be used if an invalid number smaller than 1 is given.)
    Note that, when `InitialPixels` = n^2 and `MaxSubdivide` = d, the total square grid size of pixels will then be given by $((n-1)\cdot 2^{d-1}+1)^2$.
  - `IterationPixels = int;`
    At each new integration loop, the Mesh is allowed to select `IterationPixels` squares that it wishes to subdivide; this means that at most 5×`IterationPixels` new geodesics will be integrated in each integration loop (except the first iteration); there could be less geodesics integrated if some of the pixels necessary to subdivide a square already existed previously. (Default: 100.)
  - `InitialSubdivisionToFinal = bool;`
    Normally, the Mesh selects only squares to subdivide with non-zero "weight", that is, squares where the "distance" (as determined by the value Diagnostic) is non-zero. However, if this option is set to `true`, then the Mesh will want to continue subdividing *any* square once it has been subdivided at least once (after the initial grid). (Default: `false`.)

2.6 `Integrator`

Under this subheading, the integration scheme is chosen that is used to integrate the geodesic equation, and other options set. The integration scheme is set by the option:

- `Type = "IntegratorName";`
  Options for `IntegratorName` are:
  - `RK4` (fourth-order Runge-Kutta integration scheme)

[2]The mesh implemented is a second version; the name `SquareSubdivisionMeshV2` is also accepted in config files to give the same mesh; this is to ensure correct functioning of some older config files.

– `Verlet` (velocity Verlet integration scheme)

RK4 is fourth-order while Verlet is second order, so RK4 is generally about twice as slow (since it needs to calculate twice as many Christoffel symbols). (Default: `Verlet`.)

There are other options to set here:

- `VerletVelocityTolerance = real;`
  (Only relevant if `IntegratorName` is specified to `Verlet`.) In the velocity Verlet algorithm, the velocity is updated in two steps — this setting sets how much the intermediate step velocity can differ from the final step velocity, fractionally (calculated using the Euclidean flat norm of the four-vector). If the intermediate step differs more than this tolerance level, then the last integration step to update the velocity is repeated until the tolerance is met. (Typically, this is only necessary one or two times, maximum.) See Dolence et al. (2009) eq. (14) and the discussion thereafter. If this is set to zero or negative, then there will never be any further iteration. (Default: -1.)

- `StepSize = real;`
  This sets the *basic* step size (i.e. change in affine parameter) for a single integration step. This is adjusted dynamically according to the current position and velocity of the geodesic. (Default: 0.03.)

- `SmallestPossibleStepsize = real;`
  The smallest possible step size that can be taken in one single step (after adapting `StepSize`). (Default: 10^{-12}.)

- `DerivativeH = real;`
  This is the value to use in numerical derivatives, i.e. the value h for which $f'(x)$ is approximated by $f'(x) = (f(x+h) - f(x-h))/(2h)$. (Default: 10^{-7}.)

2.7 `Output`

Under this heading, you can configure how the geodesics' will be written to file. You can also set the level of detail outputted to the screen (console) during the running of the program:

- `ScreenOutputLevel = int;`
  Here, `int` must be an integer between 0 and 4 (inclusive). 0 means no output to the screen at all except important warnings; 4 is all possible messages including any debug messages. (Default is 4.)

- `LoopMessageFrequency = int;`
  During every integration loop, every (approx.) `int` geodesics there will be a message outputted to the console to indicate the (approx.) current number of geodesics integrated, the (approx.) speed of integration (in geodesics per second), and the estimated time left in this integration loop. Note that all the numbers outputted are approximate, as they are extrapolations from the progress of one thread of the parallel integration loop to all of the threads. (Default is essentially infinite, so no intermediate output will be shown.)

The options that are available to set that determine the format of the file name(s) to which the output is written are:

- `FilePrefix = "prefix";`
  All output files will start with "prefix". If this is not specified, FOORT will do all output to the console.

- `FileExtension = "extension";`
  All output files will end with ".extension". If this is not specified or given as an empty string, the output files will have no extension.

- `TimeStamp = bool;`
  `bool` is true if every output file contains a time stamp (with the local system time and date of when FOORT starts). (Default: `true`.)

The files created will then have names such as "prefix_TIME_DiagName.extension" or "prefix_TIME_DiagName_n.extension" for $n > 1$, where "DiagName" is a short (space-less) name of the Diagnostic whose data is outputted in this file. There will be a different file for each Diagnostic turned on. n is the number of file written, which is only part of the file name for $n > 1$. The "TIME" timestamp is of the form "yymmdd-hhmmss" (year - month - day, hour - minute - second). Note that FOORT will create any directory structures necessary to create the file.

The options that further specify details of the actual output written to files are:

- `GeodesicsToCache = int;`
  Indicates how many geodesics should be cached (stored in memory) before writing everything in memory to file. By default or if 0 is specified, this is essentially infinite. (It is actually the maximum number storeable in a `unsigned long`, which is $\sim 10^9 - 10^{19}$, depending on OS).

- `GeodesicsPerFile = int;`
  Indicates how many geodesics should be written per file. If more than this number of geodesics are integrated, additional files with filenames ending in `_n` (before the specified file extension) will be created. By default or if 0 is specified, is essentially infinite (just as `GeodesicsToCache` above).

- `FirstLineInfo = bool;`
  If set to `true`, will output on the first line of every output file a string that gives descriptive data of all the objects used in integration (Metric, Source, ViewScreen, etc.) (Default: `true`.)

3 Processing Output

This section describes the output files that `FOORT` produces, and the two Python modules included in the `PostProc` folder of the repository to process them: `pyFOORT`, which turns the raw output into images, and `pyFOORTInterferometry`, which computes visibility amplitudes from `EquatorialEmission` output. Section 3.5 gives a complete worked example.

3.1 Output files

File names. `FOORT` writes a separate file for every `Diagnostic` that is turned on, named (see also the `Output` options in Section 2):

`prefix_TIME_DiagName.extension` or `prefix_TIME_DiagName_n.extension` (for $n > 1$),

where `_TIME` is only present if `TimeStamp = true`, and `.extension` only if `FileExtension` is given. The `DiagName` for each `Diagnostic` is:

`Diagnostic`	`DiagName` in file name
`FourColorScreen`	`FourColorScreen`
`GeodesicPosition`	`GeodesicPosition`
`EquatorialPasses`	`EquatPasses`
`ClosestRadius`	`ClosestRadius`
`EquatorialEmission`	`EquatorialEmission`

A new file number n is started each time `GeodesicsPerFile` geodesics have been written. Each run of `FOORT` overwrites existing files with the same name.

First line. If `FirstLineInfo = true` (the default), *every* output file, including the numbered files with $n > 1$, starts with a single line describing the run: the `Metric` (with its parameters), the `Source`, all `Diagnostics` and `Terminations` (with their options), the camera position and screen size, the `Mesh`, and the integrator settings. This line is not otherwise used by `FOORT`, but is a convenient record of how the data was produced.

Data lines. After the first line, each file contains one line per integrated geodesic, of the form

```
i j <diagnostic data>
```

where `i` and `j` are the integer pixel coordinates of the geodesic on the screen. The lines are *not* sorted: they appear in the order in which the geodesics were integrated. However, the files of the different `Diagnostics` (with the same file number n) list the geodesics in the same order, so line k refers to the same geodesic in each of them.

The pixel coordinate `i` runs along the horizontal screen direction and `j` along the vertical direction. If the screen has N_{side} pixels per side, both run from 0 to $N_{\text{side}} - 1$, and the pixels at 0 and $N_{\text{side}} - 1$ lie *exactly on* the screen edges. Pixel (i, j) thus corresponds to the screen position

$$\alpha = \alpha_c + w_x \left(\frac{i}{N_{\text{side}} - 1} - \frac{1}{2} \right), \qquad \beta = \beta_c + w_y \left(\frac{j}{N_{\text{side}} - 1} - \frac{1}{2} \right), \tag{3.1}$$

where (α_c, β_c) is `ScreenCenter` and (w_x, w_y) is `ScreenSize`. Note that the pixel spacing is $w_x/(N_{\text{side}} - 1)$. N_{side} depends on the `Mesh`:

- `SimpleSquareMesh` and `InputCertainPixelsMesh`: $N_{\text{side}} = \sqrt{\texttt{TotalPixels}}$.
- `SquareSubdivisionMesh`: the pixel coordinates are those of the *finest* possible grid, with $N_{\text{side}} = (\sqrt{\texttt{InitialPixels}} - 1) \cdot 2^{\texttt{MaxSubdivide}-1} + 1$. Only the pixels that were actually integrated appear in the file. Regions that were never refined to the finest level only contain the coarser pixels, so the output does *not* fill a regular grid. (This is why `pyFOORT` interpolates the data onto a grid; see Section 3.3.)

Diagnostic data. The data after the pixel coordinates is:

- `FourColorScreen`: a single integer: 1–4 for the quadrant of the celestial sphere in which the geodesic ended up, and 0 if it did not reach the boundary sphere.
- `EquatorialPasses`: a single integer: the number of equatorial passes, negative if the geodesic fell into the horizon.
- `ClosestRadius`: a single number: the smallest (true) r reached, which is 0 for geodesics that fell into the horizon.
- `EquatorialEmission`: two numbers: the total intensity I_{tot}, followed by the (signed) number of equatorial passes, as for `EquatorialPasses`.
- `GeodesicPosition`: `N ;; t1 r1 theta1 phi1 t2 r2 theta2 phi2 ...`, where `N` is the number of stored positions, followed by the separator `;;` and then the `N` stored positions. (Note that the r coordinates are the true radii, so r is outputted even if integration proceeded with $\log r$.)

All non-integer values are written with six digits after the decimal point. Values smaller than 5×10^{-7} are therefore written as zero, so choose emission parameters such that the relevant intensities are not this small.

If no `FilePrefix` is given, all of this output is written to the console instead (without the first line).

3.2 Setting up the Python modules

The Python modules `pyFOORT.py` and `pyFOORTInterferometry.py` are in the `PostProc` folder of the repository. They require Python ≥ 3.10 with `numpy`, `scipy`, `pandas`, `matplotlib` and `scikit-image`. The included `environment.yml` sets up a suitable `conda` environment:

```
conda env create -f environment.yml    conda activate foort-postproc
```

To use the modules from another folder, add `PostProc` to the Python path, for example:

```
import sys; sys.path.append("path/to/FOORT/PostProc")
                    import pyFOORT as pf
                import pyFOORTInterferometry as pfi
```

Every function is documented in its docstring. Running `doxygen` in the `PostProc` folder generates HTML documentation of both modules in `docs/postproc/html`. The notebook `PostProc/ExampleNotebook.ipynb` demonstrates the main functions on the output of the example configuration files (see Section 3.5).

Configuration requirements. To read `FOORT` output with these modules, keep the following in mind when setting up the `Output` options:

- The modules always look for files with extension `.dat`, so set `FileExtension = "dat";`.
- All functions take a `FilePrefix` argument, to which they append `_DiagName.dat` (or `_DiagName_n.dat`). If `TimeStamp = true`, the time stamp is part of this prefix, e.g. `"Output/MyRun_251031-120000"`. Setting `TimeStamp = false` gives simpler prefixes, at the cost of overwriting earlier runs with the same `FilePrefix`.
- If the output is split over several files, pass the number of files as `NrFiles`.
- The argument `FirstLineDescription` (default `True`) must match the `FirstLineInfo` setting of the run.

3.3 Images with `pyFOORT`

For each `Diagnostic`, `pyFOORT` has a function that reads the output file(s), puts the data on a grid, and displays the image, all in one call:

`Diagnostic`	Function
`FourColorScreen`	`FOORTToFourColorScreenImage`
`EquatorialPasses`	`FOORTToEquatorialPassesImage`
`EquatorialEmission`	`FOORTToEquatorialEmissionImage`, `FOORTToEquatorialEmissionLineout`
`ClosestRadius`	`FOORTToClosestRadius`
`GeodesicPosition`	`FOORTToDistortedBackground`

For example, `pf.FOORTToEquatorialEmissionImage("Output/MyRun", NrFiles=2)` shows the emission image of a run whose output was split over two files. These functions share the arguments:

- `FilePrefix`, `NrFiles`, `FirstLineDescription`: see Section 3.2.
- `GridFraction` (default 1): since the output of a `SquareSubdivisionMesh` does not fill a regular grid (Section 3.1), the data is first interpolated onto a regular square grid spanning the full screen, using nearest-neighbour interpolation. The grid has `GridFraction` $\cdot N_{\text{side}}$ points per side; e.g. `GridFraction = 0.5` gives a grid with half as many points per side, which is faster for large runs.

- `DisplayImageTitle` (default `False`): use the first line of the output file as the image title.
- `FileOutput`: if given, the image is also saved to this file.
- `Ax`: a `matplotlib` axes object to draw on, for example to combine several images in one figure. If not given, a new figure is created and shown.
- `Verbose` (default `False`): print progress information.

The additional arguments of each function are:

- `FOORTToFourColorScreenImage`: colours the four quadrants blue, yellow, red and green, and geodesics that did not reach the boundary sphere black. For spacetimes without a horizon, set `NoHorizon = True` to drop the black colour.
- `FOORTToEquatorialPassesImage`: shows the number of equatorial passes. Set `DiagType = "EquatorialEmission"` to take the passes from `EquatorialEmission` output instead. By default the absolute value is shown (`PlotAbsPass = True`), so geodesics that cross the equator equally often look the same whether or not they fall into the horizon. `PlotOnlyInOutBH = 0` shows only geodesics that fell into the horizon, and `PlotOnlyInOutBH = 1` only those that escaped. `PassesRange = (nmin, nmax)` sets all values outside this range to zero.
- `FOORTToEquatorialEmissionImage`: shows the total intensity I_{tot}. The options are:
  - `LimitRange = (Imin, Imax)` clips the intensities to this range. (Default: `(0.0, 100000.0)`.)
  - `TruncateRange = (Imin, Imax)` sets intensities outside this range to zero.
  - `EquatPassesRange = (nmin, nmax)` keeps only pixels whose (signed) number of equatorial passes lies in this range, and `EquatPassesSelection = [n1, n2, ...]` only pixels with one of the listed numbers of passes; if both are given, `EquatPassesSelection` is used. Since the intensity is a sum over passes, these do not isolate the contribution of a single pass; rather, they show the full intensity of the selected geodesics. For example, `EquatPassesSelection = [1]` shows only geodesics that cross the equator exactly once (the direct image), and `EquatPassesRange = (2, 1000)` shows the lensing bands.
  - `LightRingRadius`: for horizonless spacetimes with a light ring, this separates the "inner" and "outer" photon rings. The number of passes of every geodesic whose closest radius is smaller than `LightRingRadius` is given a minus sign, after which `EquatPassesRange` and `EquatPassesSelection` can select either set (e.g. `EquatPassesSelection = [-2]` for the inner $n = 2$ ring). This requires the `ClosestRadius Diagnostic` to have been turned on in the same run.
- `FOORTToEquatorialEmissionLineout`: plots the intensity along a straight line through the centre of the screen, instead of the full image. `angle` (in degrees, default 20) is measured anticlockwise from the horizontal screen axis, and must lie between $-45°$ and $45°$. `x_scale` is the full screen width in μas (default 50), used to label the distance along the line, and `ipl_points` the number of points sampled along it. With `return_lineout = True`, the function returns the distances and intensities as arrays. It takes the same selection options as `FOORTToEquatorialEmissionImage`.
- `FOORTToClosestRadius`: shows the closest radius reached, optionally on a logarithmic scale (`logscale = True`). `contours = [r1, r2, ...]` draws labelled contours at these radii; this requires `Ax` to be given. `LimitRange` and `TruncateRange` work as for the emission.

- `FOORTToDistortedBackground(FilePrefix, BackgroundFile, ...)`: shows how a background image on the celestial sphere is lensed. This uses `GeodesicPosition` output in which each geodesic has exactly one stored position, its final one: set `UpdateFrequency = 0`, `UpdateStart = false` and `UpdateFinish = true`. `BackgroundFile` is an image file in equirectangular projection, with θ from 0 to π along the vertical and ϕ from 0 to 2π along the horizontal. Pixels whose geodesic ended at a radial coordinate $\leq$ `BlackConditionRadius` (default 4.0) are drawn black.

Lower-level functions. Each `FOORTTo...` function combines three steps, which are also available separately. These are useful for custom processing, such as combining the data of several `Diagnostics`, using a different colour map, or analysing the gridded data numerically.

- `LoadFOORTRawData(FilePrefix, DiagType, NrFiles, FirstLineDescription)` reads the output file(s) of the `Diagnostic` whose file name contains `DiagType` (the `DiagName` of Section 3.1). It returns a `pandas` data frame with columns `x`, `y`, `diag1` (and `diag2` for `EquatorialEmission`), together with the first line of the file. For `GeodesicPosition`, only output with a single stored position per geodesic can be read; its columns are then `x`, `y`, `nrpts=1`, `separator=;;`, `t`, `r`, `theta` and `phi`.
- `ModifiedEquatorialEmission(FilePrefix, LightRingRadius, ...)` does the same for `EquatorialEmission` output, with the sign of the passes changed according to the `ClosestRadius` output, as described for `LightRingRadius` above.
- `DataToGrid(FOORTData, GridFraction, DiagToUse, ...)` interpolates the data frame onto a regular grid, as described for `GridFraction` above, and returns it as a `numpy` array. `DiagToUse` (1 or 2) selects the `diag1` or `diag2` column. The options `TakeAbs`, `TruncateRange`, `LimitRange`, `Diag2RangeSelect` and `Diag2AdvancedSelect` manipulate the data before gridding; the last two correspond to `EquatPassesRange` and `EquatPassesSelection` above. The first index of the array runs along the vertical screen direction, so the array is displayed correctly with `imshow(..., origin="lower")`.
- `GridToFourColorScreenImage`, `GridToEquatorialPassesImage`, `GridToEquatorialEmissionImage`, `GridToClosestRadiusImage` and `GridToDistortedImage` display a grid with the colour map and options of the corresponding `FOORTTo...` function. `DisplayImage(FOORTGrid, TheColorMap, ColorMinMax, ...)` displays a grid with any `matplotlib` colour map.

3.4 Visibility amplitudes with `pyFOORTInterferometry`

Interferometric observations (such as by the EHT) measure the Fourier transform of the image, the complex visibility

$$V(\mathbf{u}) = \int I(\mathbf{x})\, e^{-2\pi i\, \mathbf{x}\cdot\mathbf{u}}\, \mathrm{d}^2\mathbf{x}, \tag{3.2}$$

where $\mathbf{x}$ are angular coordinates on the sky (in radians) and $\mathbf{u}$ is the baseline in units of the observing wavelength. `pyFOORTInterferometry` computes the visibility amplitude $|V|$ along straight lines through the origin of the $\mathbf{u}$-plane, using the projection-slice theorem: the image is first projected onto a line at the chosen angle (its Radon transform), and the projection is then Fourier transformed in one dimension.

The main function is

```
visamps, baselines, FirstLineInfo = FOORTToVisAmp(FilePrefix, Angles, ...)
```

which reads `EquatorialEmission` output, interpolates it onto a grid (with the same `GridFraction`, `LimitRange`, `TruncateRange`, `EquatPassesRange`, `EquatPassesSelection` and `LightRingRadius` options as `FOORTToEquatorialEmissionImage`), and computes the visibility amplitudes. Its specific arguments are:

- `Angles`: a list of angles in degrees, one for each line in the **u**-plane. An angle of 0° corresponds to baselines along the horizontal screen axis, and 90° to baselines along the vertical screen axis; increasing the angle rotates the baseline direction anticlockwise (as the image is displayed by `pyFOORT`). Since the image is real, angles A and $A + 180°$ give the same amplitudes.

- `AngularSize`: the angular width of the screen in radians, i.e. the angle between the first and last pixel columns. (Default: 50 μas.) For a source of mass M at distance D, a screen width of w_x (in units of M) corresponds to $w_x\, GM/(c^2 D)$; for example, $w_x = 15M$ for M87* ($GM/c^2D \approx 3.8\,\mu$as) gives about 57 μas.

- `RadonPaddingFactor` (default 25): the projection is zero-padded to this many times its length before the Fourier transform, which sets the spacing of the baselines (see below).

- `FileOutput`: if given, the visibility amplitudes are also written to this file (with `WriteVisAmpsToFile`, see below).

It returns `visamps`, an array with one row per angle, each normalized to 1 at zero baseline, and `baselines`, the corresponding baseline lengths in Gλ (the same for all angles).

The baselines are sampled with spacing $\Delta u \approx 1/(\texttt{RadonPaddingFactor} \cdot \texttt{AngularSize})$, up to the Nyquist limit $u_{\max} = (N_{\text{grid}} - 1)/(2 \cdot \texttt{AngularSize})$, where N_{grid} is the number of grid points per side. For example, a 50 μas screen gridded with $N_{\text{grid}} = 1001$ points reaches $u_{\max} \approx 2000$ Gλ. Resolving the photon rings at long baselines thus requires a fine enough grid, i.e. enough subdivision levels in the `SquareSubdivisionMesh`.

Lower-level function. The lower-level functions are:

- `RadonTransform(FOORTEmissionGrid, Angles)`: the Radon transform of a grid, one row per angle.

- `RadonToComplexVis(FOORTRadon, PaddingFactor, sample_spacing)`: the complex visibilities from a Radon transform, at the non-negative frequencies (in cycles per unit of `sample_spacing`, i.e. per radian if `sample_spacing` is the pixel spacing in radians).

- `ComplexVisToNormVisAmp(ComplexVis)`: the visibility amplitudes, normalized to 1 at zero baseline.

- `WriteVisAmpsToFile(VisAmps, FileName, FirstLineInfo)` and `LoadVisAmpsFromFile(FileName, FirstLineDescription)`: save and load visibility amplitudes as a text file (the baselines are not saved).

- `DisplayVisAmp(VisAmp, URange, Baselines ...)`: a quick logarithmic plot of one row of visibility amplitudes against the given baselines (or against the sample index if none are given).

3.5 Example

The repository contains the configuration file `FOORT/cfgs/KerrExample.cfg`, which images a Kerr black hole with an emitting disc, with the `FourColorScreen` and `EquatorialEmission Diagnostics` turned on. Its `Output` settings are

```
FilePrefix = "Output/KerrExample"; FileExtension = "dat"; TimeStamp = false;
```

and the output is split over two files per `Diagnostic`. Running `FOORT` with this configuration file from the `FOORT` folder produces `FOORT/Output/KerrExample_FourColorScreen.dat`, `FOORT/Output/KerrExample_EquatorialEmission.dat`, and their counterparts ending in `_2.dat`. From the `PostProc` folder, the images and visibility amplitudes are then made with:

```
import matplotlib.pyplot as plt
import pyFOORT as pf
import pyFOORTInterferometry as pfi

prefix = "../FOORT/Output/KerrExample"
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
pf.FOORTToFourColorScreenImage(prefix, NrFiles=2, Ax=ax[0])
pf.FOORTToEquatorialEmissionImage(prefix, NrFiles=2, Ax=ax[1])
pf.FOORTToEquatorialPassesImage(prefix, NrFiles=2, Ax=ax[2],
                                DiagType="EquatorialEmission")
plt.show()

visamps, baselines, _ = pfi.FOORTToVisAmp(prefix, Angles=[40, 60], NrFiles=2)
plt.semilogy(baselines, visamps[0])
plt.xlabel("baseline (G$\\lambda$)")
plt.show()
```

`PostProc/ExampleNotebook.ipynb` works through this example in more detail, and compares it with the Johannsen black hole of `FOORT/cfgs/JohannsenExample.cfg`.

4 Procedures to Add New Objects

For every object to add, there are a certain number of tasks to be done. Per object, these are enumerated below. Within the source files, these add points are also indicated and sample source code is given. These steps below and the add points in the code make adding a new object not a big efFOORT at all!

A general note about variable names and settings: the name that a particular property has in the configuration file will be the same as the name that features in the first argument (in quotation marks) in the argument `LookupValue`. So, for example, the line:
`TheSetting.LookupValue("ConfigurationFilePropertyName", LocalVariableName);`
looks at the `ConfigCollection` object `TheSetting`, which will be some branch within the configuration file, and within (that branch in) the configuration file reads in the line:
`ConfigurationFilePropertyName = "mysetting";`
and puts `"mysetting"` into `LocalVariableName` (which should in this case be a `std::string`!). Note that if the setting `ConfigurationFilePropertyName` does not exist under the branch `ConfigCollection`, then `LocalVariableName` will be unaltered (no exceptions are thrown). This is why, before using `LookupValue`, `LocalVariableName` should have been set to whatever the setting's default value should be. Note that `LocalVariableName`, `ConfigurationFilePropertyName`, and whatever the class (private) member variable is called that eventually stores this property or value, can all be completely different names.

Note that `LookupValueInteger` should be used for any integer-valued settings (either signed or unsigned); `LookupValue` can be used for other settings (both floating-point numbers and strings). Also note that `LookupValue`, when called for a floating-point variable, will automatically convert integers to floats.

4.1 Adding a new Metric

A. *At the end of* `Metric.h`: Add the declaration for your new `Metric` class here, inheriting publically from the base `Metric` class (or from `SphericalHorizonMetric` if your metric has a horizon at a constant radius, and/or from `SingularityMetric` if your metric has (an arbitrary amount of) singularities (of arbitrary codimension)). It is good practice to make the class (and all overriding virtual functions) `final`, unless further descendant `Metric` classes are possible. It is also good practice to make the member variables (metric parameters) `private` and `const` (and initialized in a constructor) since the

metric should not change after initialization. The implementations (definitions) of the new metric's functions can then go in `Metric.cpp` (or a different source code file).

Necessary overrides:

- It is necessary to override the basic metric getter functions `getMetric_dd` and `getMetric_uu`, which return the metric with indices down or up. Note that these metric getters should apply the $u = \log r$ transformation if `m_rLogScale` is `true`.
- The metric should pass `rLogScale` to the base constructor (i.e. a call `Metric(rLogScale)`). If the metric descends from `SphericalHorizonMetric`, it instead should call that base class constructor and specify the horizon radius as well, i.e. a call `SphericalHorizonMetric(rh, rLogScale)` where `rh` is an expression containing the horizon radius. See the `KerrMetric` implementation for an example.

Optional overrides:

- It is also recommended to overload `getFullDescriptionStr()`, although this has been already implemented in the base class `Metric`.
- Note: if your metric enjoys any Killing vectors, make sure to remember to set `m_Symmetries` accordingly in the class constructor; e.g. a `Metric` that is stationary and axisymmetric should include the line `m_Symmetries` $= \{0, 3\}$; in the constructor.
- Optionally, you can also override the `getChristoffel_udd` function to implement e.g. an analytical Christoffel symbol calculation. (Otherwise, the default `Metric` implementation calculates Christoffel symbols using numerical derivatives of the metric.)

B. *In* `Config.cpp`*, in function* `Config::GetMetric()`: You must add a new `else if` clause checking for if your new metric type has been specified. If it has, then proceed to look up any other parameters necessary for the metric before creating a new instance of it in the variable `TheMetric`.

4.2 Adding a new Diagnostic

A.1. *In the middle of* `Diagnostics.h`*, after all other* `Diagnostic` *class declarations*: Declare your new `Diagnostic` class here, inheriting publically from `Diagnostic`. As with `Metric`, it is good practice to make this new `Diagnostic` and all its overriding functions `final`. The definitions of the member functions of your class can then be given in `Diagnostics.cpp` (or a different source code file). The necessary functions to override are:

- A constructor that passes along the `const` pointer to the (owner) `Geodesic` to the base class constructor.
- `UpdateData()`: here your `Diagnostic` updates itself according to the current state of its owner `Geodesic`.
- `getFullDataStr()`: this is the full output string of the `Diagnostic` for its owner `Geodesic`, as should be written to a file.
- `getFinalDataVal()`: this returns a vector of real numbers that indicates the final "value" that should be associated to its owner `Geodesic`.
- `FinalDataValDistance(...)`: this returns a "distance" between two geodesics based on their two final "values" (as given by `getFinalDataVal()`. This distance is used by adaptive mesh(es) to decide where to refine the mesh.

- `Reset()` *(optional)*: If your `Diagnostic` has member variables that change during the trajectory of the `Geodesic`, then `Reset()` must reset all these variables to their initial values, ready to be used for a new `Geodesic`. If there are no such member variables in your `Diagnostic`, then this function does not need an override. Note: if you do override `Reset()`, make sure to add a call `Diagnostic::Reset())` in your implementation of `Reset()`, so that the base class also resets its internal variable!
- `getNameStr()`: This returns a short string without spaces that will be appended to the file name, typically just a short name identifying the `Diagnostic`.
- `getFullDescriptionStr()`: This returns a longer, descriptive string (with spaces allowed) of the `Diagnostic` and any relevant options that are selected. This will e.g. be outputted to the screen at runtime to indicate which Diagnostics are turned on. (Note that it is not strictly necessary to override this function when implementing a new `Diagnostic`, but it is advised.)

Finally, if the `Diagnostic` needs to specify options, a declaration of its `DiagnosticOptions` should be given here. Note that this is a `std::unique_ptr` and is `static`: the reason is that these options get set once (at the start of the program), and then remain the same — i.e. for all instances of the `Diagnostic` that belong to different instances of `Geodesic`s.

A.2. *(Optional) At the end of* `Diagnostics.h`*, after all other* `DiagnosticOptions` *struct declarations*: Declare and define your new `DiagnosticOptions` struct here, if your `Diagnostic` needs additional options over the standard `UpdateFrequency` ones provided by the base struct `DiagnosticOptions`. As indicated in the code, all member variables should be `const` but `public`, and initialized in the constructor. Make sure your constructor passes along the `UpdateFrequency` struct information to the base struct constructor.

As mentioned above, the `DiagnosticOptions` are `static` members of their owning `Diagnostic` class; they get set at the beginning of the program and are afterwards immutable.

B. *At the beginning of* `Diagnostics.h` *(at the definitions of the bitflags)*: Define a new `DiagBitflag` for your new `Diagnostic`. Make sure to use a bitflag that has not been used before! (All defined bitflags are here, so it should be clear what has been used already.)

C. *At the beginning of* `Diagnostics.cpp`*, in the definition of* `CreateDiagnosticVector`: add an appropriate `if` clause checking whether diagflags contains the new DiagBitflag, and if so call `AddDiagnostic(...)`;

D.1. *At the beginning of* `Config.cpp`: If your Diagnostic carries a `static DiagnosticOptions struct` (whether it is a new descendant of `DiagnosticOptions` or not), then it must be declared here!

D.2. *In* `Config.cpp`*, in function* `Config::InitializeDiagnostics()`: Add an `if` clause here checking if your Diagnostic is turned on in the configuration file. If it is, intialize the relevant options and set the bitflags appropriately.

4.3 Adding a new Termination

A.1. *At the end of the declarations of the* `Termination` *classes in* `Terminations.h`: Declare your new `Termination` class here, inheriting publically from `Termination`. The definitions of the member functions of your class can then be given in `Terminations.cpp` (or a different source code file). The necessary functions to override are:

- A constructor that passes along the `const` pointer to the (owner) `Geodesic` to the base class constructor.
- `CheckTermination()`: which checks if the termination condition has been reached; if not, returns `Term::Continue`, if so, returns the new termination condition (see below in B.2.).
- `Reset()` *(optional)*: If your `Termination` has member variables that change during the trajectory of the `Geodesic`, then `Reset()` must reset all these variables to their initial values, ready to be used for a new `Geodesic`. If there are no such member variables in your `Termination`, then this function does not need an override. Note: if you do override `Reset()`, make sure to add a call `Termination::Reset())` in your implementation of `Reset()`, so that the base class also resets its internal variable!
- `getFullDescriptionStr()`: This returns a descriptive string (with spaces allowed) of the `Termination` and any relevant options that are selected. This will e.g. be outputted to the screen at runtime to indicate which Terminations are turned on.

Finally, most likely the `Terminations` needs to specify options, so a declaration of its `TerminationOptions` should be given here. Note that this is a `std::unique_ptr` and is `static`: as for `DiagnosticOptions`, these options get set once (at the start of the program), and then remain the same — i.e. for all instances of the `Termination` that belong to different instances of `Geodesic`s.

A.2. *(Optional) At the end of* `Terminations.h`: Declare and define your new `TerminationOptions` struct here, if necessary, inheriting publically from `TerminationOptions`. Just as for `DiagnosticOptions`, all member variables should be `const` but `public`, and initialized in the constructor. Make sure your constructor passes along the number of steps to the base struct `TerminationOptions`.

B.1. *At the beginning of* `Terminations.h` *(at the definitions of the bitflags)*: Define a new `TermBitflag` for your new `Termination`.

B.2. *In the definition of* `enum class Term` *in* `Terminations.h`: Add a new termination condition in this class that can be set by your new `Termination`.

C. *At the beginning of* `Terminations.cpp`*, in the definition of* `CreateTerminationVector`: add an appropriate `if` clause checking to see if `termflags` contains the newly defined `TermBitflag` (which was defined in point B.1 above), and if so adds an instance of the `Termination` to the `Termination` vector being created.

D.1. *At the beginning of* `Config.cpp`: If your Termination carries a `static TerminationOptions struct` (whether it is a new descendant of `TerminationOptions` or not), then it must be declared here!

D.2. *In* `Config.cpp`*, in function* `Config::InitializeTerminations()`: Add an `if` clause here checking if your Termination is turned on in the configuration file. If it is, intialize the relevant options and set the bitflags appropriately.